Lab_Thread_Counter — Static vs Non-Static Counter in Multithreading

Name: Sanjith Saha | **ID:** IT24044 | **Course:** SEPM

1. Files

- `Sanjith_Thread.java` – typed version (for running)
- `outputs/` – raw `.txt` output of every run and `results.csv`
- `run_all.ps1` / `run_all.sh` – run all test cases (5 unsynchronized runs per thread count) and write `outputs/results.csv`
- `show_cases.ps1` / `show_cases.sh` – print the output of one test case (use it for screenshots)

2. How to run

```
javac Sanjith_Thread.java
java Sanjith_Thread 10 100000 true    # thread-safe (AtomicLong)
java Sanjith_Thread 10 100000 false   # unsynchronized
```

Run all test cases: `powershell -ExecutionPolicy Bypass -File .\run_all.ps1` (Windows) or `bash run_all.sh` (Linux/macOS).

3. Test inputs

Test	Threads	Increments/thread	Expected
TC1	1	1,000	1,000
TC2	2	10,000	20,000
TC3	5	10,000	50,000
TC4	10	50,000	500,000
TC5	20	50,000	1,000,000
TC6	50	50,000	2,500,000
TC7	100	50,000	5,000,000

4. Output screenshots

Screenshots are from separate runs of the program (`bash show_cases.sh <TC number>`, Linux, 1 CPU core). The unsynchronized values change between runs, so they differ from the tables in Section 5 (see Section 6).

TC1 (1 thread, 1,000 increments; safe mode first, then unsynchronized)

```
user@linux:~/Lab_Thread_Counter$ bash show_cases.sh 1

===== TC1: java Sanjith_Thread 1 1000 true =====
Mode            : Thread-safe (AtomicLong)
Threads         : 1
Increments/thr  : 1000
Expected count  : 1000
Static count    : 1000
Non-static total: 1000
Absolute diff   : 0
Difference (%)  : 0.0000

===== TC1: java Sanjith_Thread 1 1000 false =====
Mode            : Unsynchronized (long)
Threads         : 1
Increments/thr  : 1000
Expected count  : 1000
Static count    : 1000
Non-static total: 1000
Absolute diff   : 0
Difference (%)  : 0.0000
```

TC2 (2 threads, 10,000 increments; safe mode first, then unsynchronized)

```
user@linux:~/Lab_Thread_Counter$ bash show_cases.sh 2

===== TC2: java Sanjith_Thread 2 10000 true =====
Mode            : Thread-safe (AtomicLong)
Threads         : 2
Increments/thr  : 10000
Expected count  : 20000
Static count    : 20000
Non-static total: 20000
Absolute diff   : 0
Difference (%)  : 0.0000

===== TC2: java Sanjith_Thread 2 10000 false =====
Mode            : Unsynchronized (long)
Threads         : 2
Increments/thr  : 10000
Expected count  : 20000
Static count    : 20000
Non-static total: 20000
Absolute diff   : 0
Difference (%)  : 0.0000
```

TC3 (5 threads, 10,000 increments; safe mode first, then unsynchronized)

```
user@linux:~/Lab_Thread_Counter$ bash show_cases.sh 3

===== TC3: java Sanjith_Thread 5 10000 true =====
Mode            : Thread-safe (AtomicLong)
Threads         : 5
Increments/thr  : 10000
Expected count  : 50000
Static count    : 50000
Non-static total: 50000
Absolute diff   : 0
Difference (%)  : 0.0000

===== TC3: java Sanjith_Thread 5 10000 false =====
Mode            : Unsynchronized (long)
Threads         : 5
Increments/thr  : 10000
Expected count  : 50000
Static count    : 50000
Non-static total: 50000
Absolute diff   : 0
Difference (%)  : 0.0000
```

TC4 (10 threads, 50,000 increments; safe mode first, then unsynchronized)

```
user@linux:~/Lab_Thread_Counter$ bash show_cases.sh 4

===== TC4: java Sanjith_Thread 10 50000 true =====
Mode            : Thread-safe (AtomicLong)
Threads         : 10
Increments/thr  : 50000
Expected count  : 500000
Static count    : 500000
Non-static total: 500000
Absolute diff   : 0
Difference (%)  : 0.0000

===== TC4: java Sanjith_Thread 10 50000 false =====
Mode            : Unsynchronized (long)
Threads         : 10
Increments/thr  : 50000
Expected count  : 500000
Static count    : 150000
Non-static total: 500000
Absolute diff   : 350000
Difference (%)  : 70.0000
```

TC5 (20 threads, 50,000 increments; safe mode first, then unsynchronized)

```
user@linux:~/Lab_Thread_Counter$ bash show_cases.sh 5

===== TC5: java Sanjith_Thread 20 50000 true =====
Mode            : Thread-safe (AtomicLong)
Threads         : 20
Increments/thr  : 50000
Expected count  : 1000000
Static count    : 1000000
Non-static total: 1000000
Absolute diff   : 0
Difference (%)  : 0.0000

===== TC5: java Sanjith_Thread 20 50000 false =====
Mode            : Unsynchronized (long)
Threads         : 20
Increments/thr  : 50000
Expected count  : 1000000
Static count    : 1000000
Non-static total: 1000000
Absolute diff   : 0
Difference (%)  : 0.0000
```

TC6 (50 threads, 50,000 increments; safe mode first, then unsynchronized)

```
user@linux:~/Lab_Thread_Counter$ bash show_cases.sh 6

===== TC6: java Sanjith_Thread 50 50000 true =====
Mode            : Thread-safe (AtomicLong)
Threads         : 50
Increments/thr  : 50000
Expected count  : 2500000
Static count    : 2500000
Non-static total: 2500000
Absolute diff   : 0
Difference (%)  : 0.0000

===== TC6: java Sanjith_Thread 50 50000 false =====
Mode            : Unsynchronized (long)
Threads         : 50
Increments/thr  : 50000
Expected count  : 2500000
Static count    : 2500000
Non-static total: 2500000
Absolute diff   : 0
Difference (%)  : 0.0000
```

TC7 (100 threads, 50,000 increments; safe mode first, then unsynchronized)

```
user@linux:~/Lab_Thread_Counter$ bash show_cases.sh 7

===== TC7: java Sanjith_Thread 100 50000 true =====
Mode             : Thread-safe (AtomicLong)
Threads          : 100
Increments/thr   : 50000
Expected count   : 5000000
Static count     : 5000000
Non-static total: 5000000
Absolute diff    : 0
Difference (%)   : 0.0000

===== TC7: java Sanjith_Thread 100 50000 false =====
Mode             : Unsynchronized (long)
Threads          : 100
Increments/thr   : 50000
Expected count   : 5000000
Static count     : 4950000
Non-static total: 5000000
Absolute diff    : 50000
Difference (%)   : 1.0000
```

5. Result analysis

Thread-safe (AtomicLong)

Threads	Increments/thread	Expected	Static	Non-static	Abs diff	Diff %
1	1,000	1,000	1,000	1,000	0	0.0000
2	10,000	20,000	20,000	20,000	0	0.0000
5	10,000	50,000	50,000	50,000	0	0.0000
10	50,000	500,000	500,000	500,000	0	0.0000
20	50,000	1,000,000	1,000,000	1,000,000	0	0.0000
50	50,000	2,500,000	2,500,000	2,500,000	0	0.0000
100	50,000	5,000,000	5,000,000	5,000,000	0	0.0000

Unsynchronized (5 runs each)

Threads	Run	Expected	Static (unsafe)	Non-static	Abs diff	Diff %
1	1	1,000	1,000	1,000	0	0.00
1	2	1,000	1,000	1,000	0	0.00
1	3	1,000	1,000	1,000	0	0.00
1	4	1,000	1,000	1,000	0	0.00
1	5	1,000	1,000	1,000	0	0.00
2	1	20,000	20,000	20,000	0	0.00
2	2	20,000	20,000	20,000	0	0.00
2	3	20,000	20,000	20,000	0	0.00
2	4	20,000	20,000	20,000	0	0.00
2	5	20,000	20,000	20,000	0	0.00
5	1	50,000	50,000	50,000	0	0.00

Threads	Run	Expected	Static (unsafe)	Non-static	Abs diff	Diff %
5	2	50,000	50,000	50,000	0	0.00
5	3	50,000	50,000	50,000	0	0.00
5	4	50,000	50,000	50,000	0	0.00
5	5	50,000	50,000	50,000	0	0.00
10	1	500,000	236,581	500,000	263,419	52.68
10	2	500,000	500,000	500,000	0	0.00
10	3	500,000	455,388	500,000	44,612	8.92
10	4	500,000	310,416	500,000	189,584	37.92
10	5	500,000	500,000	500,000	0	0.00
20	1	1,000,000	750,000	1,000,000	250,000	25.00
20	2	1,000,000	1,000,000	1,000,000	0	0.00
20	3	1,000,000	1,000,000	1,000,000	0	0.00
20	4	1,000,000	1,000,000	1,000,000	0	0.00
20	5	1,000,000	1,000,000	1,000,000	0	0.00
50	1	2,500,000	2,500,000	2,500,000	0	0.00
50	2	2,500,000	2,500,000	2,500,000	0	0.00
50	3	2,500,000	2,500,000	2,500,000	0	0.00
50	4	2,500,000	2,500,000	2,500,000	0	0.00
50	5	2,500,000	2,384,062	2,500,000	115,938	4.64
100	1	5,000,000	5,000,000	5,000,000	0	0.00
100	2	5,000,000	5,000,000	5,000,000	0	0.00
100	3	5,000,000	4,800,000	5,000,000	200,000	4.00
100	4	5,000,000	5,000,000	5,000,000	0	0.00
100	5	5,000,000	4,972,927	5,000,000	27,073	0.54

Average of 5 unsynchronized runs

Threads	Avg static	Avg abs diff	Avg Diff %	Min %	Max %
1	1,000	0	0.00	0.00	0.00
2	20,000	0	0.00	0.00	0.00
5	50,000	0	0.00	0.00	0.00
10	400,477	99,523	19.90	0.00	52.68
20	950,000	50,000	5.00	0.00	25.00
50	2,476,812	23,188	0.93	0.00	4.64
100	4,954,585	45,415	0.91	0.00	4.00

6. Observation & Conclusion

> **Note:** These numbers were produced on a **single-CPU-core** machine, where threads are time-sliced and rarely run truly in parallel, so lost updates were much rarer than on a multi-core PC. Re-run `run_all` on your own computer to get your own results; the tables above are regenerated from `outputs/results.csv`.

Observations

Thread-safe experiment (AtomicLong). In all seven test cases (1 to 100 threads) the static counter matched the non-static total exactly. The absolute difference was 0 and the percentage difference was 0%. This is the control case and confirms the program counts correctly when the shared counter is updated atomically.

Unsynchronized experiment (regular `long`). The non-static total was always exactly correct, because each thread increments its own private counter. The static counter was sometimes lower than expected (for 10, 20, 50 and 100 threads in these runs), which means increments were lost. With 1 thread no loss is possible.

Threads	Average difference (%)
1	0.00
2	0.00
5	0.00
10	19.90
20	5.00
50	0.93
100	0.91

- **1 thread:** no loss (0%), because only one thread touches the counter.
- **Run-to-run variation:** for the same thread count, results changed a lot between runs (for example, 10 threads gave 0% in two runs and up to 52.68% in another). Thread scheduling is decided by the OS and JVM and is not repeatable.
- **Trend with thread count:** the percentage difference does not grow steadily with more threads; it depends on scheduling and on how many CPU cores can really run threads at the same time.

Why this happens

`count++` on a normal `long` is not one operation. It is three steps: read the value, add 1, write it back. If two threads read the same value before either writes, both write the same result and one increment is lost. This is a race condition. `AtomicLong.incrementAndGet()` performs the whole update as one indivisible operation (compare-and-set), so no update is lost. The non-static counter is never shared, so it needs no protection.

Conclusion

- `static` versus non-static describes **ownership** (one copy for the class versus one copy per object). It does not make a variable thread-safe.
- A shared static counter needs synchronization or atomic operations; otherwise concurrent updates can be lost.
- The thread-safe experiment gave 0% difference in every case, as expected.
- The unsynchronized result is unpredictable: it can be correct in one run and wrong in the next.

7. Analysis questions (answers)

1. Static belongs to the class (one copy); non-static belongs to each object.
2. Because there is only one copy per class, every thread accesses the same memory.
3. Each thread gets its own object, so its own `instanceCount` copy.
4. `join()` makes `main` wait; otherwise the totals are read before the threads finish.
5. Lost updates from the read-modify-write race on `count++`.
6. No – it depends on scheduling and hardware; the trend is not guaranteed to be monotonic.
7. Different thread interleavings on each run.
8. `AtomicLong` uses atomic CAS, so each increment is indivisible and no update is lost.
9. Create one object and pass the same object to all `Threads` (and make its counter `AtomicLong` or synchronized).